2026

C Workshop

Control Structures in C

Welcome to SESSION 2



Wameedh SC

Agenda

1 Conditional statement

1. Understanding Conditions & Expressions
2. Relational + logical operators.
3. The if Statement – Basic Decision Making
4. if…else – Two-Way Decisions
5. else if Chains – Multi-Path Decisions
6. Nested Conditions
7. Mini exercise

2 Loops

1. Why Loops? Understanding Repetition
2. The for Loop — Counting & Iteration
3. The while Loop — Condition-Based Iteration
4. The do…while Loop — Execute at Least Once
5. Loop Control: break & continue

3 Switch statement

1. Structure of switch statement
2. How case & break Work
3. The default Case
4. When to Use switch vs if…else
5. Examples

1*

Conditional statements

1- Understanding Conditions & Expressions:

"Before we can use if statements or loops, we need to understand how C evaluates conditions. A condition is simply an expression that results in either true or false. In C, true = any non-zero value and false = 0."

1. What is a Condition?

A condition is an expression that C evaluates to decide whether to execute a block of code.

Examples of conditions:

```
x > 10
```

```
age == 18
```

```
number != 0
```

```
(a + b) > c
```

These expressions always result in true (non-zero) or false (0).

2- Relational + logical operators.

Operator	Meaning	Example
==	Equal to	a==b returns 1 if a and b are the same
>	Greater than	a>b returns 1 if a is larger than b
<	Less than	a<b returns 1 if a is smaller than b
>=	Greater than or equal to	a>=b returns 1 if a is larger than or equal to b
<=	Less than or equal to	a<=b returns 1 if a is smaller than or equal to b
!=	Not equal to	a!=b returns 1 if a and b not the same

&& **AND**

```
x=6
y=3
x<10 && y>1 Return True
```

|| **OR**

```
x=6
y=3
x==5 || y==5 Return False
```

! **NOT**

```
x=6
y=3
!(x==y) Return True
```

EXAMPLE

```c
int main() {
    int temperature;
    int isSunny;

    printf("Enter the temperature: ");
    scanf("%d", &temperature);

    printf("Is it sunny? (1 = yes, 0 = no): ");
    scanf("%d", &isSunny);

    // Both conditions must be true
    if (temperature > 30 && isSunny == 1) {
        printf("It's hot AND sunny! Turning on the air conditioning...\n");
    }
```

3- The if Statement – Basic Decision Making

```
if (condition) {
    // code to execute if condition is true
}
```

EXAMPLE:

```
int age = 20;

if (age >= 18) {
    printf("You are an adult.\n");
}
```

Make your own statement NOW!!

4- if…else – Two-Way Decisions

```
if (condition) {
    // true case
} else {
    // false case
}
```

EXAMPLE:

```
int number = 5;

if (number % 2 == 0) {
    printf("Even number\n");
} else {
    printf("Odd number\n");
}
```

Make your own statement NOW!!

5- else if Chains – Multi-Path Decisions

```
if (condition1) {
    // runs if condition1 is true
} else if (condition2) {
    // runs if condition1 is false AND condition2 is true
} else if (condition3) {
    // runs if both previous conditions are false
} else {
    // runs if none of the conditions are true
}
```

EXAMPLE:

```
int score;

printf("Enter your score: ");
scanf("%d", &score);

if (score >= 90) {
    printf("Grade: A\n");
} else if (score >= 80) {
    printf("Grade: B\n");
} else if (score >= 70) {
    printf("Grade: C\n");
} else if (score >= 60) {
    printf("Grade: D\n");
} else {
    printf("Grade: F\n");
}
```

6- Nested Conditions

```
if (condition1) {
    if (condition2) {
        // runs only if BOTH condition1 and condition2 are true
    }
}
```



EXAMPLE:

```
int isCold = 1;
int isRaining = 1;

if (isCold) {
    if (isRaining) {
        printf("Wear a jacket and take an umbrella.\n");
    } else {
        printf("Wear a jacket.\n");
    }
} else {
    printf("The weather is good! No jacket needed.\n");
}
```

2*

Loops ∞ ∞ ∞

1- Why Loops? Understanding Repetition

"Loops allow a program to repeat actions without writing the same code many times. Instead of copying a line 10 or 100 times, a loop can run it automatically."

Would you write this 100 times?

```c
printf("1\n");
printf("2\n");
printf("3\n");
printf("4\n");
printf("5\n");
```

SOLUTION:

```c
int i;

for (i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
```

2- The for Loop — Counting & Iteration

```
for (initialization; condition; update) {
    // repeated code
}
```

— Common mistakes that we all do:

- Using < instead of <=
- Forgetting the update (i++)
- Infinite loops with wrong condition
- Starting at 0 vs starting at 1

EXAMPLES:

```
for (int i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
```

```
for (int i = 0; i < 3; i++) {
    printf("Hello!\n");
}
```

Write this so i can give you a remark

3- The while Loop — Condition-Based Iteration

"A while loop repeats as long as a condition is true. We use it when we don't know in advance how many times we will repeat something."

EXAMPLE : "Keep eating until you are full."

Important points:
The condition is checked before each repetition ✓

If the condition is false at the start → loop never runs ✓

You must update something inside the loop or it becomes infinite ✓

```
while (condition) {
    // repeated code
}
```

SIMPLE EXAMPLE

```
int i = 1;

while (i <= 5) {
    printf("%d\n", i);
    i++;
}
```

Practical Example

```
int password = 0;
while (password != 1234) {
    printf("Enter password: ");
    scanf("%d", &password);
}
printf("Access granted!\n");
```

4- The do...while Loop — Execute at Least Once

"The do–while loop is special. It runs the code first, then checks the condition. This means the loop will run at least once, no matter what."

```
do {
    // code runs at least once
} while (condition);
```

EXAMPLE:

```
int i = 1;

do {
    printf("%d\n", i);
    i++;
} while (i <= 5);
```

4- The do...while Loop — Execute at Least Once

User MUST enter a password at least one time

```c
int password;

do {
    printf("Enter password: ");
    scanf("%d", &password);
} while (password != 1234);

printf("Access granted!\n");
```

Here, the code will run once, because it will not check the condition until it's finished

(Example of previous while loop)

```c
int password = 0;

while (password != 1234) {
    printf("Enter password: ");
    scanf("%d", &password);
}

printf("Access granted!\n");
```

Here, the code will run once, because the first condition is correct

5- Loop Control: break & continue

Sometimes we want to stop a loop early, or skip one step.
C gives us two special commands to control loops:
break → stops the loop immediately
continue → skips to the next iteration

1 - The break Statement (Stop the Loop)

"Ends the loop instantly, even if the condition is still true".

```c
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    printf("%d\n", i);
}
```

1 2 3 4

```c
int num;
while (1) { // infinite loop
    printf("Enter a number (0 to exit): ");
    scanf("%d", &num);
    if (num == 0) {
        break; // stop loop
    }
    printf("You entered: %d\n", num);
}
```

2-The continue Statement (Skip One Step)

“Skips the rest of the loop body, moves to the next iteration”

```c
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    printf("%d\n", i);
}
```

1 2 4 5

```c
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue; // skip even numbers
    }
    printf("%d ", i);
}
```

1 3 5 7 9

3*

Switch case

1- Structure of a Switch Statement

"When you have many possible choices, using many else if statements becomes long and messy.
The switch statement makes multi-choice decisions easier and cleaner."

```
switch (expression) {
    case value1:
        // code
        break;

    case value2:
        // code
        break;

    default:
        // code if no case matches
}
```

Important points!!

- expression must be int or char (not float)
- break prevents fall-through
- default is optional but recommended

2- How case & break Work

2-1 Case

Inside a switch, each case is like a label.

When the value of the switch expression matches a case, the program jumps directly to that case.

```
switch (x) {
    case 2:
        printf("This will run if x == 2\n");
        break;

    case 5:
        printf("This will run if x == 5\n");
        break;
}
```

If x is 5 → program jumps straight to case 5.

2- How case & break Work

2-2 Break

break means:

- Stop the switch immediately.
- Jump out of the entire switch block.

Without break, the program keeps running into the next case (this is called fall-through).

```c
int x = 2;

switch (x) {
    case 1:
        printf("One\n");
        break;

    case 2:
        printf("Two\n");
        break;

    case 3:
        printf("Three\n");
        break;
}
```

3- The default Case

What is the default Case in a switch?

- The default case is the “catch-all” option in a switch statement.
- It runs when none of the cases match the value of the switch expression.
- Think of it like:
- “If nothing else matches, do this.”

Why is default important?

- Prevents unexpected behavior when the user enters a value you didn’t plan for
- Makes the program more safe and error-proof
- Helps beginners understand complete decision-making
- Works like the else in an if–else chain
- Without default, invalid input might do nothing — confusing for the user.

3- The default Case

The default is placed at the bottom, the end of all cases

```c
int day = 10;

switch (day) {
    case 1:
        printf("Monday\n");
        break;
    case 2:
        printf("Tuesday\n");
        break;
    default:
        printf("Invalid day\n");
}
```

```
Invalid day
```

SIMPLE EXAMPLE

```c
int day = 3;

switch (day) {
    case 1:
        printf("Monday\n");
        break;
    case 2:
        printf("Tuesday\n");
        break;
    case 3:
        printf("Wednesday\n");
        break;
    default:
        printf("Invalid day\n");
}
```

GAME MENU

```c
int choice;
printf("1. Play\n");
printf("2. Settings\n");
printf("3. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
    case 1:
        printf("Starting game...\n");
        break;
    case 2:
        printf("Opening settings...\n");
        break;
    case 3:
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid option\n");
}
```

Why switch case and not else if statement?

```c
int day = 3;
switch (day) {
    case 1:
        printf("Monday\n");
        break;
    case 2:
        printf("Tuesday\n");
        break;
    case 3:
        printf("Wednesday\n");
        break;
    default:
        printf("Invalid day\n");
}
```

```c
int day = 3;
if (day == 1) {
    printf("Monday\n");
}
else if (day == 2) {
    printf("Tuesday\n");
}
else if (day == 3) {
    printf("Wednesday\n");
}
else {
    printf("Invalid day\n");
}
```

You can see the difference right ?

IMPORTANT EXAMPLE

```c
char grade = 'B';
switch (grade) {
    case 'A':
    case 'B':
    case 'C':
        printf("You passed!\n");
        break;
    case 'D':
    case 'F':
        printf("You failed.\n");
        break;
    default:
        printf("Invalid grade.\n");
}
```

NOTE

Multiple cases can share the same code

Important notes about the switch statement

All Switch-Case Problems:

- Missing break → fall-through
- Using unsupported types (float, string, ranges)
- Using complex conditions in cases
- Duplicate case labels
- Missing default
- Using switch instead of else-if (wrong situation)
- Unintentional fall-through
- Case values must be constants
- Very long, messy switch structures
- Forgetting to loop the menu

Switch is BAD when:

- You need ranges (x <= 10)
- You need logical operators
- You need non-integer comparisons
- You need greater/less than

HOMEWORK

Write small C programs to practice control structures:

(1) Ask the user for a number and print whether it's positive, negative, or zero using if–else

(2) Ask for a score (0–100) and print "Excellent", "Good", "Pass", or "Fail" using an else-if chain

(3) Ask if it's cold and if it's raining (1 = yes, 0 = no) and give clothing advice using nested if statements

(4) Use a for loop to print numbers 1 to 5

(5) Use a while loop to count down from 5 to 1

(6) Use a do-while loop to keep asking the user to "Enter 1 to stop" until they enter 1

(7) Use a for loop with continue to print numbers 1–5 but skip 3

(8) Create a simple menu using a switch: 1 prints "Hello", 2 prints "Goodbye", and any other number prints "Invalid choice."

Send your codes to my email : yanisboub06@gmail.com

Thank you for your attention

Powered by Wameedh SC



Yanis Boubekeur